

1.5 Unit Vector



Core skills you will master in this section:

- ▶ Vector normalization: the standard way to turn any nonzero vector into a unit vector, by dividing it by its L2 norm.
- ▶ The relationship between 2D unit vectors and the unit circle: a 2D unit vector's head lies on the unit circle.
- ▶ The relationship between 3D unit vectors and the unit sphere: a 3D unit vector's head lies on the surface of the unit sphere.
- ▶ Expressing a vector as “length × direction vector”: a form that separates magnitude from direction.
- ▶ Polar coordinates: describing a vector's position in 2D with a radius and an angle, and converting between Cartesian and polar coordinates.
- ▶ Spherical coordinates: how to convert between spherical and 3D Cartesian coordinates.

Unit vectors play an important role in topics such as the inner product, scalar projection, vector projection, polar coordinates, spherical coordinates, orthonormal systems, orthonormal bases, eigenvalue decomposition, and singular value decomposition; for this reason, we devote an entire section to introducing unit vectors.

Normalizing a Nonzero Vector

Vector normalization is a way of rescaling a nonzero vector so that its length becomes 1 while its direction stays the same.

Vector normalization is a special case of scalar multiplication of a vector.

Normalizing any nonzero n -dimensional column vector $\mathbf{a}$ means multiplying the vector by the reciprocal of its length (magnitude, L^2 norm, modulus, or Euclidean norm), that is,

$$\hat{\mathbf{a}} = \frac{\mathbf{a}}{\|\mathbf{a}\|} = \frac{1}{\sqrt{a_1^2 + a_2^2 + \cdots + a_n^2}} \begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_n \end{bmatrix}$$

The result $\hat{\mathbf{a}}$ is called the direction vector, i.e., the unit vector, also known as the normalized vector. When emphasizing direction, this book usually uses the term direction vector.

The direction vector $\hat{\mathbf{a}}$ points in the same direction as the original vector $\mathbf{a}$.

The direction vector $\hat{\mathbf{a}}$ has length 1, that is, $\|\hat{\mathbf{a}}\| = 1$.

Placing the normalized versions of different nonzero vectors side by side makes it easy to compare their directions visually. In particular, once we have the inner product of vectors, we will see that the inner product of two unit vectors is exactly the cosine of the angle between them; that is the topic of the next section. Normalization also shows why scalar multiplication never changes a vector's direction (only its length and, for negative scalars, its sign): a and any positive multiple of a normalize to the very same unit vector.

⚠ Note that the zero vector $\mathbf{0}$ is usually considered to have no direction, since its magnitude is zero and no unique direction can be determined; however, in some mathematical contexts, the zero vector $\mathbf{0}$ can also be regarded as parallel to every direction.

2D Unit Vectors

For any nonzero 2D vector $\mathbf{a} = [a_1, a_2]^T$, its unit vector is

$$\hat{\mathbf{a}} = \frac{\mathbf{a}}{\|\mathbf{a}\|} = \begin{bmatrix} \frac{a_1}{\sqrt{a_1^2 + a_2^2}} \\ \frac{a_2}{\sqrt{a_1^2 + a_2^2}} \end{bmatrix}$$

For example, given the 2D column vector $\mathbf{a} = [3, 4]^T$, its unit vector is

$$\hat{\mathbf{a}} = \frac{\mathbf{a}}{\|\mathbf{a}\|} = \frac{1}{\sqrt{3^2 + 4^2}} \times \begin{bmatrix} 3 \\ 4 \end{bmatrix} = \begin{bmatrix} 3/5 \\ 4/5 \end{bmatrix}$$

As shown in Figure 1, after $\mathbf{a} = [3, 4]^T$ is normalized, its length becomes 1 while its direction stays the same.

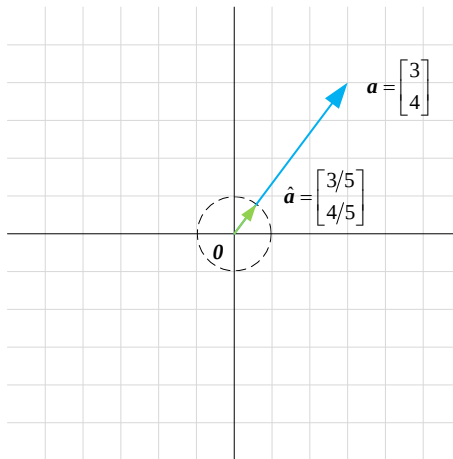


Figure 1. Normalizing the 2D Vector $\mathbf{a}$

As shown in Figure 2, once a 2D nonzero vector is normalized, if its tail is at the origin, its head lands exactly on the unit circle.

Clearly, $\mathbf{e}_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$ and $\mathbf{e}_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$ are also unit vectors, representing the positive direction of the x_1 axis and the positive direction of the x_2 axis, respectively; while $-\mathbf{e}_1 = \begin{bmatrix} -1 \\ 0 \end{bmatrix}$ represents the negative direction of the x_1 axis, and $-\mathbf{e}_2 = \begin{bmatrix} 0 \\ -1 \end{bmatrix}$ represents the negative direction of the x_2 axis. The heads of all four of these vectors also lie on the unit circle.

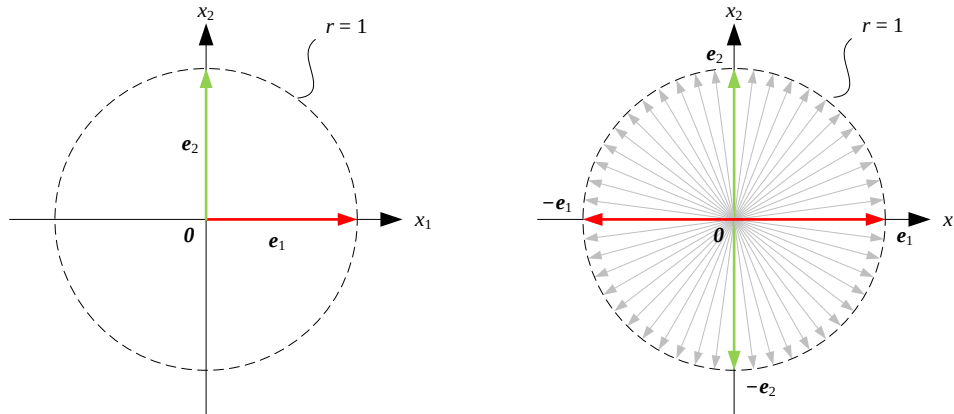


Figure 2. Unit Vectors in the Plane, with Tails at the Origin and Heads on the Unit Circle

Length × Direction Vector

Conversely, a nonzero vector $\mathbf{a}$ can be written in the form “length × direction vector,” namely

$$\mathbf{a} = \underbrace{\|\mathbf{a}\|}_{\text{Length}} \underbrace{\hat{\mathbf{a}}}_{\text{Direction}}$$

where $\|\mathbf{a}\|$ supplies the vector's magnitude (length, modulus, L2 norm, Euclidean norm), and the direction vector $\hat{\mathbf{a}}$ supplies its direction.

For example, the 2D column vector $\mathbf{a} = [3, 4]^T$ can be written as

$$\begin{bmatrix} 3 \\ 4 \end{bmatrix} = 5 \times \begin{bmatrix} 3/5 \\ 4/5 \end{bmatrix}$$

Here, 5 is the vector's length, and $[3/5, 4/5]^T$ is the direction vector (unit vector).

Code 1 computes a unit vector; please study it on your own. In addition, please modify the code to check whether a vector $\mathbf{a}$ is the zero vector.

Code 1. Unit Vector |  LA_01_05_01.ipynb

```

## 初始化
import numpy as np

## 定义向量 a
a = np.array([3, 4])

## 计算向量的长度
norm_a = np.linalg.norm(a)

## 计算单位向量
unit_a = a / norm_a

## 长度  $\times$  方向向量
norm_a * unit_a

```



The Unit Circle and Concentric Circles

As shown in Figure 3, when a vector's tail is at the origin: if its length (magnitude, L^2 norm, Euclidean norm, modulus) is less than 1 (blue), its head lands inside the unit circle; if the vector's length is greater than 1 (red), its head lands outside the unit circle.

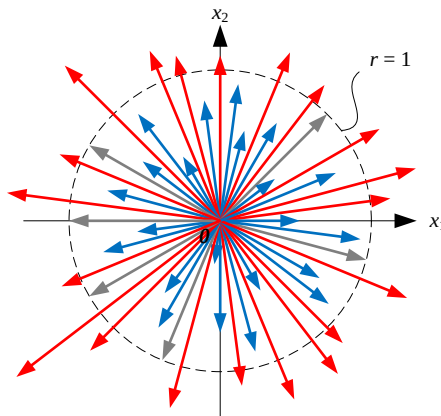


Figure 3. Vectors Whose Heads Lie on, Inside, and Outside the Unit Circle

As shown in Figure 4, every vector starting from the origin with the same length (say r) has its head lying on the circle of radius r centered at that length.

When vectors have different lengths, their heads land on circles of different radii, forming a family of concentric circles.

We will explore this further in the section on vector norms later in this chapter.

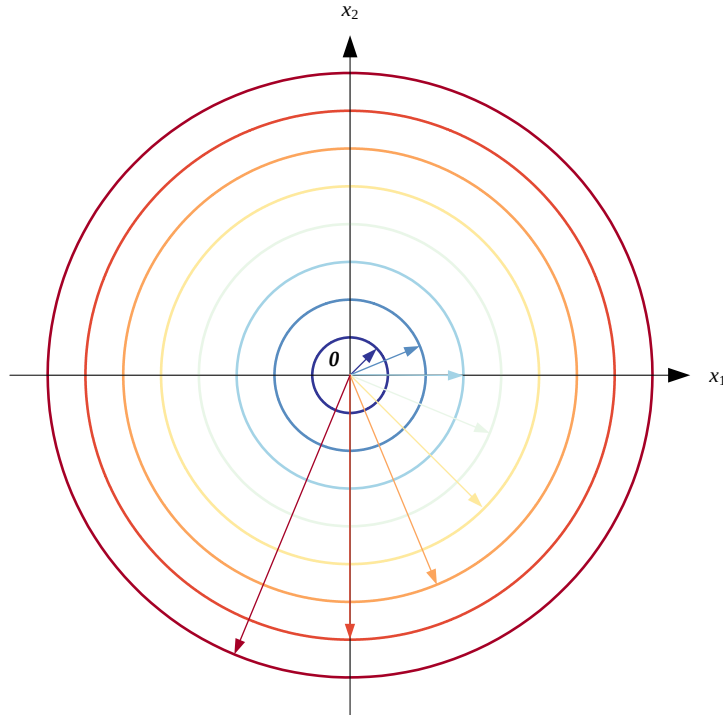


Figure 4. Concentric Circles for Vectors of Different Lengths

Projection

In the Cartesian coordinate system, the horizontal coordinate a_1 is, in fact, the projection of the vector $\mathbf{a} = \begin{bmatrix} a_1 \\ a_2 \end{bmatrix}$ in the horizontal direction, while the vertical coordinate a_2 is the projection of $\mathbf{a}$ in the vertical direction, and the vector's length is $\|\mathbf{a}\|$.

This projection is, in fact, the scalar projection that we will introduce later in this chapter.

Simply put, the scalar projection is the length (a scalar, or coordinate value) of one vector's projection onto the direction of another vector.

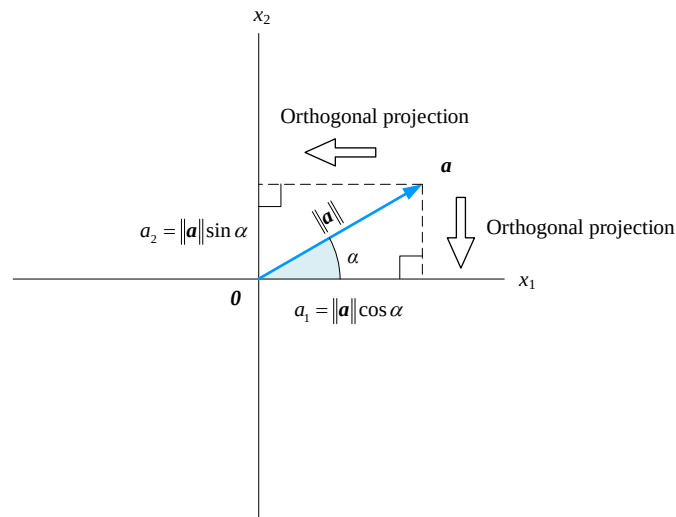


Figure 5. Scalar Projections of a Vector Onto the Horizontal and Vertical Directions

It's like the sun shining straight down at noon: the shadow of a vertical pole falls straight onto the horizontal ground, and the shadow's length is the pole's orthogonal projection onto the ground.

As shown in Figure 5, using the Pythagorean theorem, we obtain

$$\begin{cases} a_1 = \|\mathbf{a}\| \cos \alpha \\ a_2 = \|\mathbf{a}\| \sin \alpha \end{cases}$$

As shown in Figure 6 (a), sine is the ratio of the opposite side's length to the hypotenuse's length in the right triangle associated with an angle, which can also be understood as the vertical coordinate of the corresponding point on the unit circle.

As shown in Figure 6 (b), cosine is the ratio of the adjacent side's length to the hypotenuse's length in the right triangle associated with an angle, which can also be understood as the horizontal coordinate of the corresponding point on the unit circle. Figure 6 also shows several other commonly used trigonometric functions.

α is the angle between the vector $\mathbf{a}$ and the positive horizontal direction, that is,

$$\alpha = \arctan\left(\frac{a_2}{a_1}\right)$$

Here, arctan is the arctangent, the inverse function of the tangent (tan for short); simply put, it gives the angle corresponding to a known tangent value.

As shown in Figure 6 (c), the tangent $\tan \alpha$ is the ratio of the opposite side's length to the adjacent side's length in the right triangle associated with an angle α , which can also be understood as the ratio of the vertical coordinate to the horizontal coordinate of the corresponding point on the unit circle.

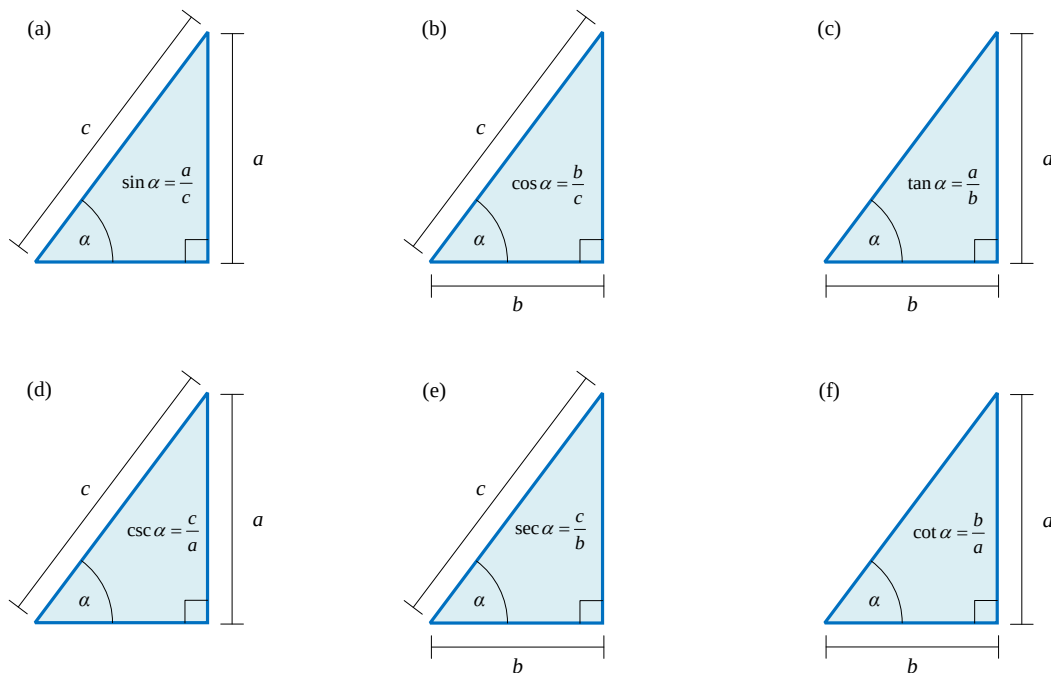


Figure 6. Common Trigonometric Functions

In this way, the column vector $\mathbf{a}$ can be written in the form “length $\times$ direction vector”

$$\mathbf{a} = \begin{bmatrix} a_1 \\ a_2 \end{bmatrix} = \begin{bmatrix} \|\mathbf{a}\| \cos \alpha \\ \|\mathbf{a}\| \sin \alpha \end{bmatrix} = \|\mathbf{a}\| \begin{bmatrix} \cos \alpha \\ \sin \alpha \end{bmatrix}$$

$\|\mathbf{a}\|$ is a scalar representing the vector's length, while $\begin{bmatrix} \cos \alpha \\ \sin \alpha \end{bmatrix}$ is the direction vector (unit vector) representing the direction of $\mathbf{a}$.

Polar Coordinates

This, in fact, gives us another way of describing the position of a point in the plane — polar coordinates.

That is, the coordinates of a point in the plane can be expressed by a length (the radial distance, r) and a direction (the polar angle, θ).

The origin of the polar coordinate system is called the pole.

The polar axis is a reference ray starting from the pole, usually coinciding with the positive half of the horizontal axis in the Cartesian coordinate system.

The radial distance is the distance from a given point to the pole, usually denoted r . For a vector $\mathbf{a}$, r corresponds to $\|\mathbf{a}\|$.

The polar angle is the angle between the radial distance and the polar axis, usually denoted θ ; θ is typically measured in radians, with counterclockwise taken as positive, generally ranging over $[0, 2\pi)$.

In the polar coordinate system, a point's position is written as (r, θ) . As shown in Figure 7, Cartesian coordinates and polar coordinates can be converted into each other.

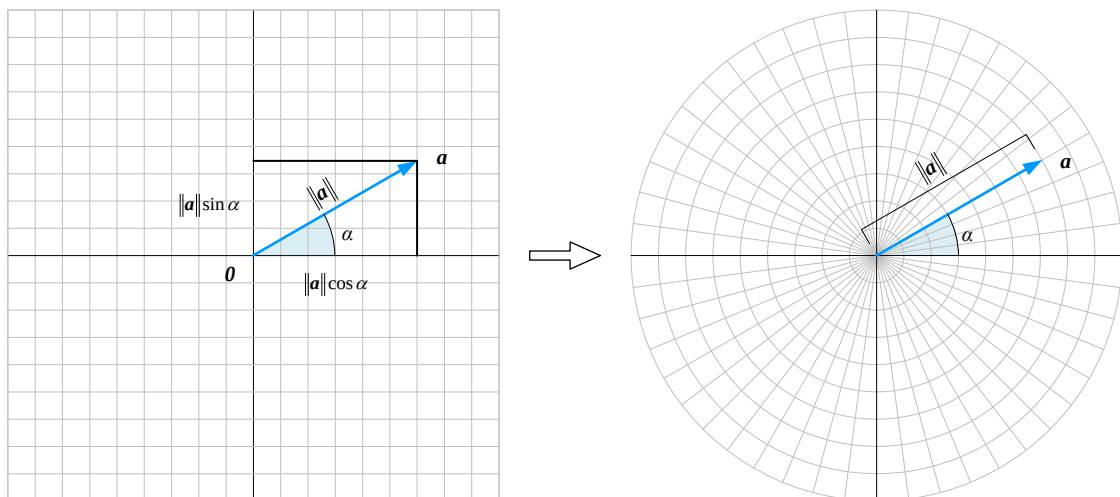


Figure 7. From the Cartesian Plane to Polar Coordinates

Code 2 converts between Cartesian and polar coordinates in the plane. Let's walk through the key lines below.

This PDF file is the author's draft, published to make it convenient for readers to study on mobile devices; the final content is subject to the printed edition published by Tsinghua University Press.

Copyright belongs to Tsinghua University Press. Please do not use it commercially, and please credit the source when quoting.

Code and PDF file downloads: <https://github.com/Visualize-ML>.

The companion video lectures for this book are all published on Bilibili — 生姜 DrGinger: <https://space.bilibili.com/513194466>

Comments and corrections are welcome. The book's dedicated email address: jiang.visualize.ml@gmail.com

- a This line computes the radial distance; `numpy.sqrt()` is the square root function, which returns the square root of its input. `x1**2` denotes the square of `x1`, and `x2**2` denotes the square of `x2`.
- b This uses `numpy.arctan2()` to compute the arctangent, with the result in radians. Unlike `numpy.arctan()`, `numpy.arctan2()` correctly handles angles in all quadrants, without needing to separately account for a negative `x`.
- c This uses `numpy.rad2deg()` to convert the input value from radians to degrees.

Code 2. Converting Between Cartesian and Polar Coordinates | LA_01_05_02.ipynb

```
## 初始化
import numpy as np

## 平面直角坐标转化为极坐标
x1, x2 = 3, 4

# 计算极径
a r = np.sqrt(x1**2 + x2**2)

# 计算极角
b theta = np.arctan2(x2, x1)
c np.rad2deg(theta)

## 极坐标转化为平面直角坐标
x1_ = r * np.cos(theta)
x2_ = r * np.sin(theta)
```

Drawing a Circle

Code 3 shows how to draw a unit circle in the 2D Cartesian coordinate system using Python, as shown in Figure 8. Visualizing a Circle (a).

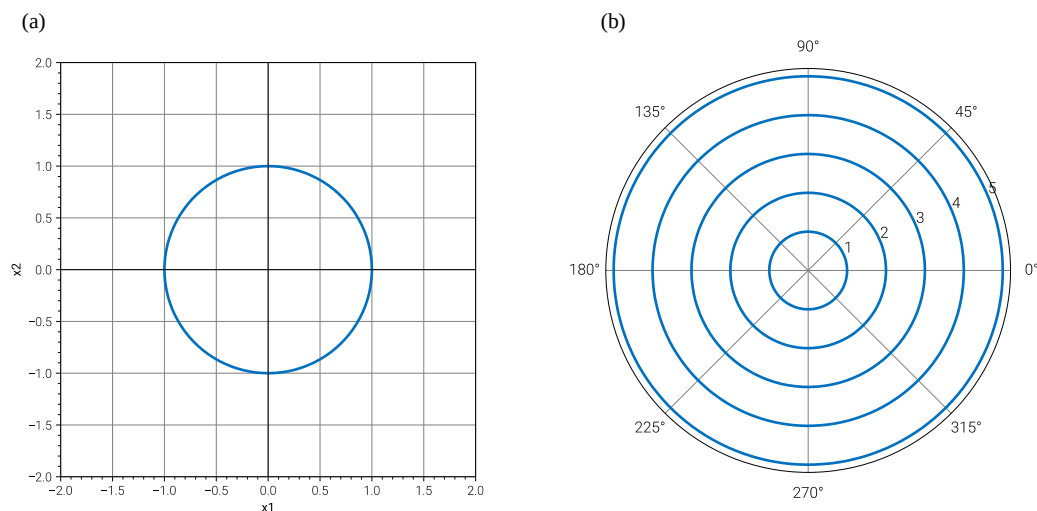


Figure 8. Visualizing a Circle

Let's walk through the key lines below.

a First, this generates an array called `theta_array`.

`numpy.linspace(a, b, n)` means taking `n` evenly spaced numbers from the interval `a` to `b`. Here, that means taking 300 evenly spaced points from 0 to $2 * \text{numpy.pi}$, i.e., across one full revolution of the circle.

`numpy.pi` is the constant π provided by NumPy, so $2 * \text{np.pi}$ is a full revolution expressed in radians. This array represents all the angles from 0 to one full turn.

`numpy.cos()` is the cosine function provided by NumPy; taking the `theta_array` generated earlier as input, it produces a new array `x1_array`, whose values are the horizontal coordinates of points on the unit circle.

Similarly, `numpy.sin()` is the sine function, producing the array `x2_array`, i.e., the vertical coordinates of points on the unit circle.

b This uses `matplotlib.pyplot.subplots()` (`plt.subplots()` for short) to create a figure canvas `fig` and an axes object `ax`. We can draw all kinds of plots on the axes object `ax`. `figsize=(6,6)` means the canvas size is 6 inches by 6 inches, ensuring the plotted region is a square.

c On the axes object `ax`, `ax.plot()` draws a line plot. The first two arguments are the horizontal and vertical coordinate data, here the horizontal and vertical coordinates of the 300 points on the circle computed earlier. `'b'` is the abbreviation for blue. `linewidth=1` means the line thickness is 1, a relatively thin line. In the end, it connects these points in sequence, which looks like the outline of a circle (though zooming in reveals it's actually a polyline).

d These are various plot decorations; let's go through them one by one.

`ax.set_aspect('equal')` sets the scale of the axes so that the horizontal and vertical axes use the same unit length. With this line added, the circle will actually look round instead of being stretched out of shape.

`ax.grid(True)` displays gridlines on the plot, like the background of graph paper.

`ax.set_xlim([-2, 2])` sets the display range of the horizontal axis, meaning it extends from -2 on the left to 2 on the right.

`ax.set_ylim([-2, 2])` is similar to the previous line, but sets the display range of the vertical axis. It also runs from -2 to 2, keeping the plot a square view.

`ax.axhline(0, color='k', linewidth=0.5)` draws a horizontal line, i.e., the x-axis. The first argument, 0, means the line is drawn at $y=0$; `color='k'` means the line is black, and `linewidth=0.5` means the line width is 0.5.

`ax.axvline(0, color='k', linewidth=0.5)` is similar to the previous line, but draws a vertical line, i.e., the y-axis. The first argument, 0, means a black line is drawn at $x=0$, with a line width of 0.5.

`ax.set_xlabel('x1')` adds a label to the horizontal axis, reading 'x1'.

`ax.set_ylabel('x2')` adds a label to the vertical axis, reading 'x2'.

Code 3. Drawing a Unit Circle in the 2D Cartesian Coordinate System |  LA_01_05_03.ipynb

```

## 初始化
import numpy as np
import matplotlib.pyplot as plt

## 生成数据
a theta_array = np.linspace(0, 2 * np.pi, 300) # 0 到 2π 之间均匀取点
x1_array = np.cos(theta_array)
x2_array = np.sin(theta_array)

## 可视化
b fig, ax = plt.subplots(figsize=(6,6))

# 绘制单位圆
c ax.plot(x1_array, x2_array, 'k', linewidth=1) # 黑色圆, 线宽 1

# 设置 XY 轴比例相同
d ax.set_aspect('equal')
ax.grid(True)

# 设置坐标轴范围
ax.set_xlim([-2, 2])
ax.set_ylim([-2, 2])

# 添加坐标轴
ax.axhline(0, color='k', linewidth=0.5)
ax.axvline(0, color='k', linewidth=0.5)

# 增加轴标签
ax.set_xlabel('x1')
ax.set_ylabel('x2')

```



LA_01_05_04.ipynb shows how to represent multiple concentric circles using the polar coordinate system; please study it on your own.

3D Unit Vectors

For any nonzero 3D vector $\mathbf{a} = [a_1, a_2, a_3]^T$, its direction vector is

$$\hat{\mathbf{a}} = \frac{\mathbf{a}}{\|\mathbf{a}\|} = \begin{bmatrix} \frac{a_1}{\sqrt{a_1^2 + a_2^2 + a_3^2}} \\ \frac{a_2}{\sqrt{a_1^2 + a_2^2 + a_3^2}} \\ \frac{a_3}{\sqrt{a_1^2 + a_2^2 + a_3^2}} \end{bmatrix}$$

This PDF file is the author's draft, published to make it convenient for readers to study on mobile devices; the final content is subject to the printed edition published by Tsinghua University Press.

Copyright belongs to Tsinghua University Press. Please do not use it commercially, and please credit the source when quoting.

Code and PDF file downloads: <https://github.com/Visualize-ML>

The companion video lectures for this book are all published on Bilibili — 生姜 DrGinger: <https://space.bilibili.com/513194466>

Comments and corrections are welcome. The book's dedicated email address: jiang.visualize.ml@gmail.com

For example, given the 3D column vector $\mathbf{b} = [2, 3, 6]^T$, its direction vector is

$$\hat{\mathbf{b}} = \frac{\mathbf{b}}{\|\mathbf{b}\|} = \frac{1}{\sqrt{2^2 + 3^2 + 6^2}} \times \begin{bmatrix} 2 \\ 3 \\ 6 \end{bmatrix} = \begin{bmatrix} 2/7 \\ 3/7 \\ 6/7 \end{bmatrix}$$

The 3D column vector $\mathbf{b} = [2, 3, 6]^T$ can also be written in the form “length $\times$ direction vector,” namely

$$\mathbf{b} = 7 \times \begin{bmatrix} 2/7 \\ 3/7 \\ 6/7 \end{bmatrix} = \begin{bmatrix} 2 \\ 3 \\ 6 \end{bmatrix}$$

If a 3D direction vector (unit vector) has its tail at the origin, its head lies on the surface of the unit sphere.

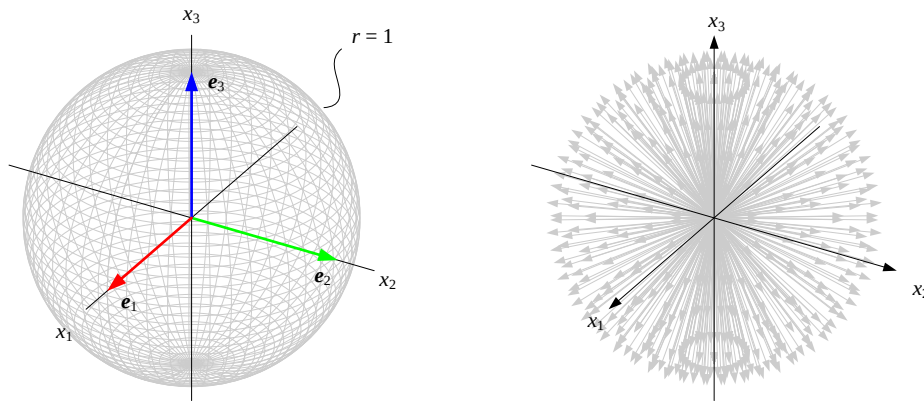


Figure 9. In 3D Space, Unit Vector Heads Lie on the Surface of the Unit Sphere

Figure 10 shows unit vectors of length 1 in RGB space, including the familiar basis vectors: the pure red vector $\mathbf{e}_1$, pure green vector $\mathbf{e}_2$, and pure blue vector $\mathbf{e}_3$.

All of these vectors have their tails at the origin, and their heads are distributed over one-eighth of the surface of the unit sphere of radius 1.

However, the vector corresponding to white, $[1, 1, 1]^T$, has length $\sqrt{3}$, which exceeds the unit sphere's radius. So the white vector does not lie on the unit sphere's surface — it lies outside it. More generally, the n -dimensional all-ones vector has length $\sqrt{n}$, so only in one dimension is an all-ones vector already a unit vector.

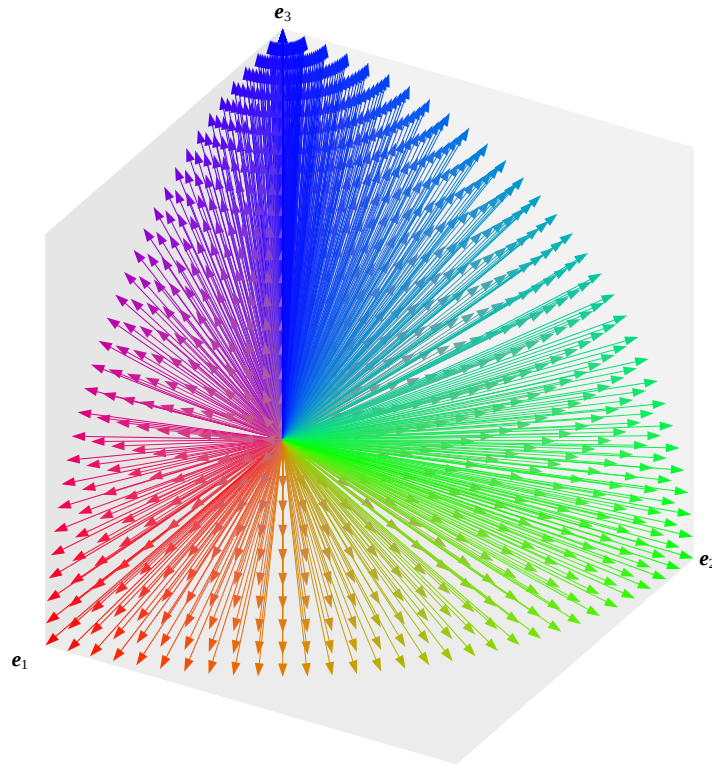


Figure 10. Unit Vectors in RGB Color Space

Spherical Coordinates

Unit vectors in the three-dimensional Cartesian coordinate system are closely related to the spherical coordinate system.

As shown in Figure 11, spherical coordinates can be thought of as being built from two planar polar coordinate systems.

A point P in the spherical coordinate system is located using the spherical coordinates (r, θ, φ) .

Here, r is the distance between P and the origin O , also called the radial distance;

θ is the angle between the line OP and the positive x_3 axis, called the polar angle; the spherical coordinate θ ranges over $[0, \pi]$. This angle is analogous to Earth's "latitude angle."

The projection of the line OP onto the x_1x_2 plane is the line OH , and φ is the angle between OH and the positive x_1 axis, called the azimuth angle. The spherical coordinate φ ranges over $[0, 2\pi]$. This angle is analogous to Earth's "longitude angle."

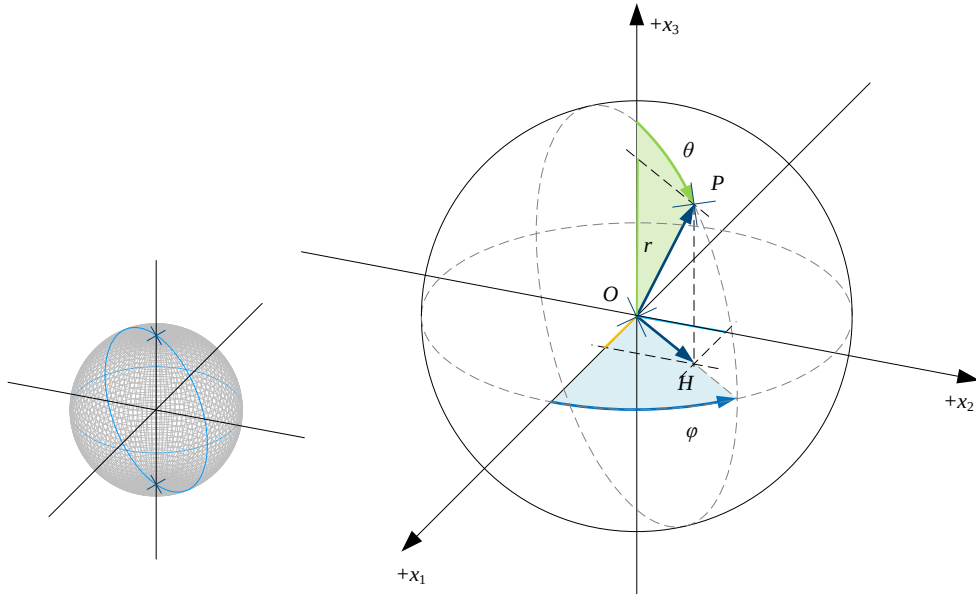


Figure 11. The Spherical Coordinate System, image from 《The Beauty of Visualization》

Next, let's look at the conversion between spherical coordinates (r, θ, φ) and 3D Cartesian coordinates (x_1, x_2, x_3) .

As shown in Figure 12 (b), since x_3 is perpendicular to the x_1x_2 plane, the projection of r onto x_3 is exactly P 's coordinate value along the x_3 axis, that is,

$$x_3 = r \cos \theta$$

As shown in Figure 12 (c), the projection of PO onto the x_1x_2 plane is

$$r \sin \theta$$

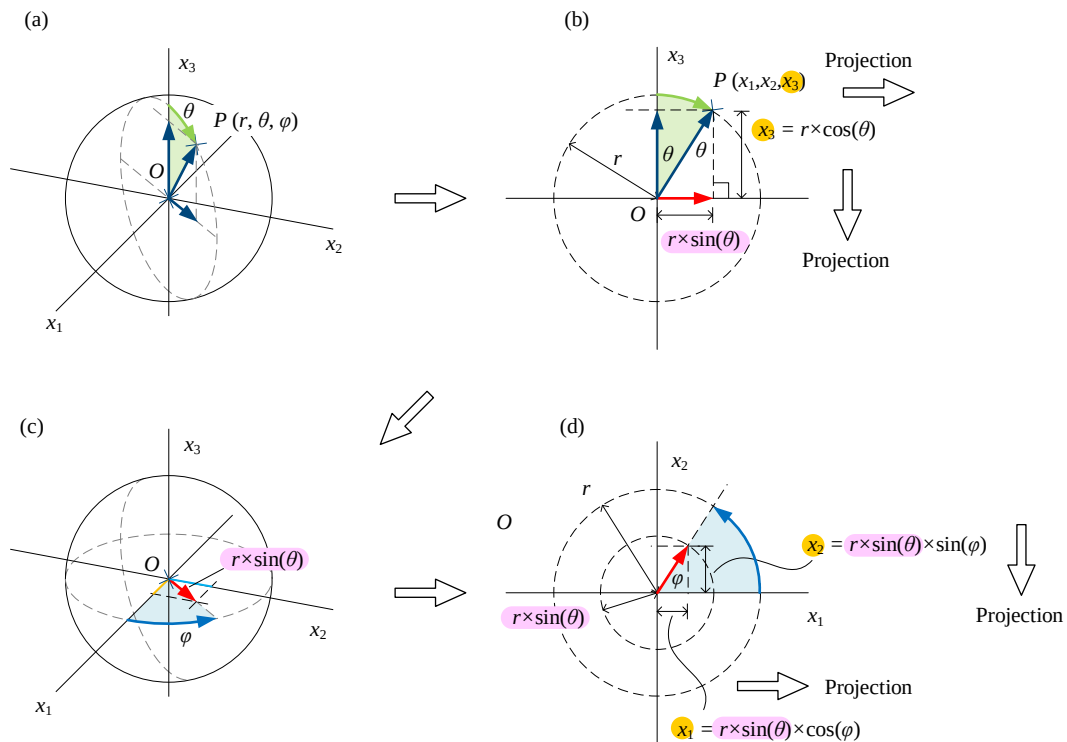


Figure 12. From Spherical Coordinates to 3D Cartesian Coordinates, image from 《The Beauty of Visualization》

Then, as shown in Figure 12 (d), the projection of $r \sin \theta$ onto the x_1 axis is P's coordinate value along the x_1 axis

$$x_1 = r \sin \theta \cos \varphi$$

the projection of $r \sin \theta$ onto the x_2 axis is P's coordinate value along the x_2 axis

$$x_2 = r \sin \theta \sin \varphi$$

Putting all this together, we obtain P's coordinates in the 3D Cartesian coordinate system:

$$\begin{cases} x_1 = r \sin \theta \cos \varphi \\ x_2 = r \sin \theta \sin \varphi \\ x_3 = r \cos \theta \end{cases}$$

In this way, the vector corresponding to point P is

$$\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} r \sin \theta \cos \varphi \\ r \sin \theta \sin \varphi \\ r \cos \theta \end{bmatrix} = r \begin{bmatrix} \sin \theta \cos \varphi \\ \sin \theta \sin \varphi \\ \cos \theta \end{bmatrix}$$

and its corresponding direction vector is

$$\begin{bmatrix} \sin \theta \cos \varphi \\ \sin \theta \sin \varphi \\ \cos \theta \end{bmatrix}$$

Conversely, using the Pythagorean theorem, r is

$$r = \sqrt{x_1^2 + x_2^2 + x_3^2}$$

This is exactly the length of the vector.



LA_01_05_05.ipynb completes the conversion between 3D Cartesian and spherical coordinates; please study it on your own.

Visualizing the Unit Sphere

Code 4 draws the unit sphere's wireframe mesh shown in Figure 13 (a).

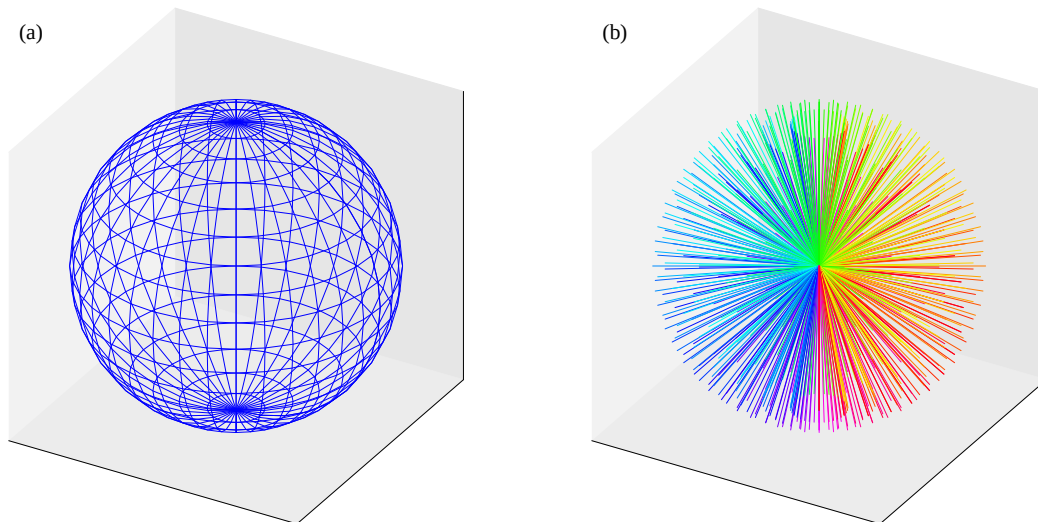


Figure 13. Visualizing the Unit Sphere

Let's walk through the key lines below.

a First, `numpy.linspace()` generates 37 evenly spaced points from 0 to 2π . This array represents the “longitude angle,” the angle swept out going once around the globe.

Then `numpy.linspace()` generates 19 points from 0 to π . This is the “latitude angle,” representing the half-circle from the North Pole to the South Pole.

The `numpy.meshgrid()` function generates a 2D grid. `u` and `v` are two 1D arrays, and this function expands them into 2D arrays `uu` and `vv`, so that we can compute over a 2D grid of angles — effectively pairing every point on the sphere with a longitude angle and a latitude angle.

b This converts each spherical coordinate on the sphere's surface into x_1 , x_2 , and x_3 axis coordinates.

c This creates a figure object and adds a 3D subplot to it. 111 means “the 1st subplot in a 1-row-by-1-column grid,” and `projection='3d'` means this plot is three-dimensional.

d `plot_wireframe()` is the function used to draw the mesh; the `xx`, `yy`, `zz` passed in are the positions of each grid point. `color='blue'` means the line color is blue, and `linewidth=0.5` means the line width is 0.5.



LA_01_05_06.ipynb also plots Figure 13 (b); please study it on your own.

Code 4. Drawing the Unit Sphere in 3D Space |  LA_01_05_06.ipynb

```

## 初始化
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.cm import hsv

## 创建球面数据
a [ u = np.linspace(0, 2 * np.pi, 37) # 生成经度角
    v = np.linspace(0, np.pi, 19) # 生成纬度角
    uu, vv = np.meshgrid(u, v) # 生成网格
    r = 1 # 球半径

# 计算球面坐标
b [ xx = r*np.cos(uu) * np.sin(vv)
    yy = r*np.sin(uu) * np.sin(vv)
    zz = r*np.cos(vv)

## 可视化
c [ fig = plt.figure(figsize=(6, 6))
    ax = fig.add_subplot(111, projection='3d')

# 绘制球面网格线
d [ ax.plot_wireframe(xx, yy, zz, color='blue', linewidth=0.5)

# 设置坐标轴范围
ax.set_xlim([-1, 1])
ax.set_ylim([-1, 1])
ax.set_zlim([-1, 1])

# 隐藏坐标轴刻度
ax.set_xticks([])
ax.set_yticks([])
ax.set_zticks([])
ax.set_aspect('equal', 'box')
ax.set_proj_type('ortho')

```



Please use tools such as DeepSeek/ChatGPT to complete the following exercises for this section.

Q1. Annotate the example below line by line, and learn how to draw a contour plot.

https://matplotlib.org/stable/gallery/images_contours_and_fields/contour_demo.html

Q2. Learn to draw a filled contour plot.

https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.contourf.html

Q3. Please write Python code to plot the sine and cosine curves.

Q4. Convert the Cartesian point (4, 4) into polar coordinates.

Q5. Compute the unit vector of $[4, 4]^T$; and write $[4, 4]^T$ in the form “length $\times$ direction vector.”

Q6. Compute the unit vector of $[1, 2, 2]^T$; and write $[1, 2, 2]^T$ in the form “length $\times$ direction vector.”

This PDF file is the author's draft, published to make it convenient for readers to study on mobile devices; the final content is subject to the printed edition published by Tsinghua University Press.

Copyright belongs to Tsinghua University Press. Please do not use it commercially, and please credit the source when quoting.

Code and PDF file downloads: <https://github.com/Visualize-ML>

The companion video lectures for this book are all published on Bilibili — 生姜 DrGinger: <https://space.bilibili.com/513194466>

Comments and corrections are welcome. The book's dedicated email address: jiang.visualize.ml@gmail.com

Q7. Convert the polar coordinates $(2, \pi/3)$ into Cartesian coordinates.

Q8. Convert the spherical coordinates $(4, \pi/6, \pi/4)$ into 3D Cartesian coordinates.